ARQUITECTURA DE COMPUTADORES

2º curso, Grado en Ingeniería Informática-Ingeniería del Software

Departamento de Arquitectura y Tecnología de Computadores

Universidad de Sevilla

PARALELISMO EN UN PROCESADOR

1. OBJETIVOS

En esta práctica se pretende estudiar un procesador segmentado, comprobar el efecto de los bloqueos de datos y control sobre el rendimiento, así como estudiar algunas técnicas para reducir estos bloqueos. Para ello, utilizaremos el simulador VisualRISCV32. Se trata de un simulador de un procesador segmentado RISC-V. Con esta práctica se pretende que el alumno:

- consolide sus conocimientos sobre segmentación de procesadores
- utilice simuladores para evaluar el rendimiento de un procesador
- conozca y aplique algunas de las técnicas para mejorar el rendimiento en procesadores segmentados

2. REQUISITOS

El alumno antes de asistir al laboratorio debe:

- estudiar la arquitectura y juego de instrucciones del procesador RISC-V
- conocer los conceptos relacionados con la segmentación de procesadores
- haber asimilado las consideraciones expuestas sobre el simulador VisualRISCV32

3. ALGUNAS CONSIDERACIONES SOBRE VisualRISCV32

VisualRISCV32 es un simulador que emula completamente un procesador RISC-V de 32 bits, tanto la cadena de ejecución, como los contenidos de la memoria, el fichero de registros y los registros temporales. Además, consta de varios parámetros configurables por el usuario.

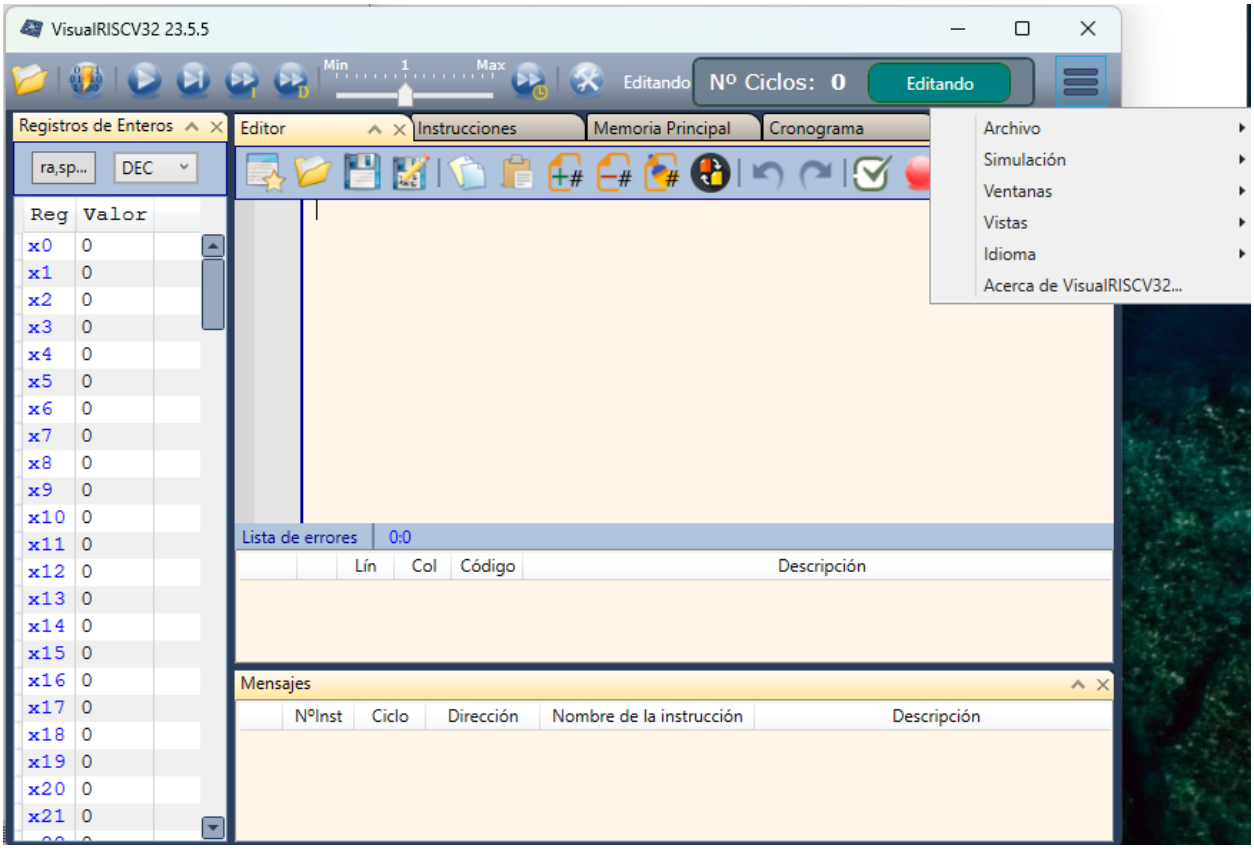


Figura 1

La figura 1 muestra el aspecto de la ventana principal de usuario del simulador VisualRISCV32. En la parte superior izquierda se encuentran los botones del menú de ejecución. Para la configuración de las propiedades de ejecución dispone de distintas ventanas en las que se detalla toda la información relativa a la ejecución de las instrucciones y que pueden ser activadas o desactivadas por el usuario. Para activar/desactivar las ventanas se debe usar el submenú ventanas dentro del menú desplegable situado en la esquina superior derecha tal como se muestra en la figura 1.
Entre las ventanas que se pueden activar podemos destacar:

- *Ventana de Editor.* Ventana de entrada de fichero en lenguaje ensamblador.
- *Ventana de Instrucciones*. Muestra información detallada (dirección de memoria, código de instrucción y nemotécnico) sobre el programa que ha sido cargado tal como se muestra en la Figura 2. Adicionalmente muestra el estado de ejecución de cada instrucción (etapa de pipelining) y resultado de la ejecución. Haciendo doble *clic* sobre una instrucción puede obtenerse información completa sobre la codificación en binario y el formato de la instrucción.
- *Ventana de Memoria Principal*. Muestra el contenido de la memoria principal (en la zona izquierda) y de las memorias caché (instrucciones y datos) correspondientes (en la zona derecha). La Figura 3 muestra el contenido de la memoria principal de una zona de memoria donde está almacenado código junto con la correspondiente caché de instrucciones. La Figura 4 muestra un segmento de memoria de datos junto con el contenido de la caché de datos.

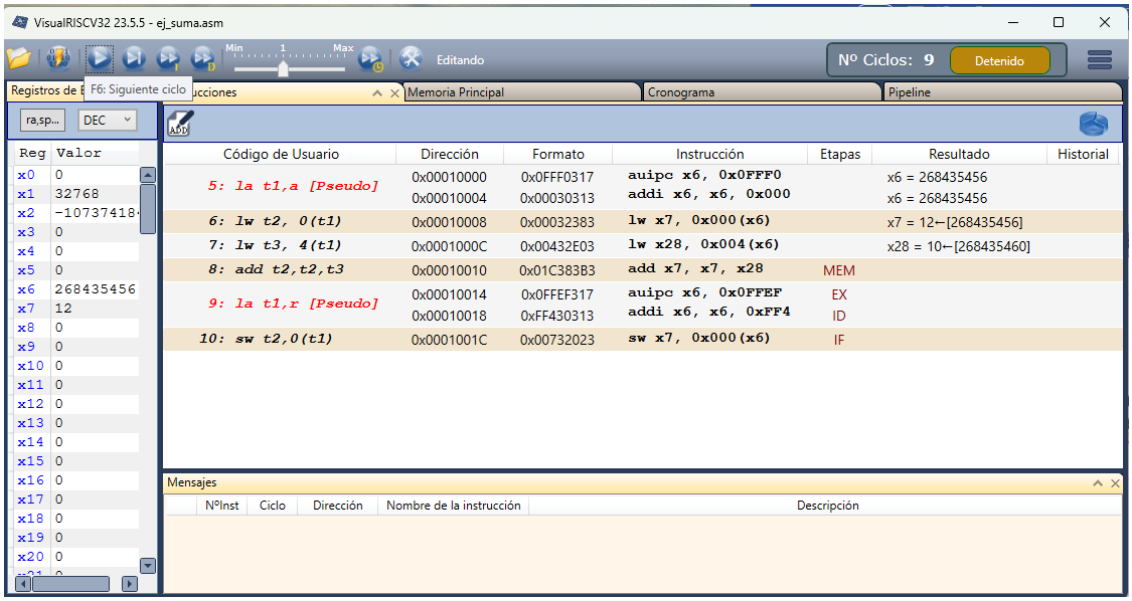


Figura 2

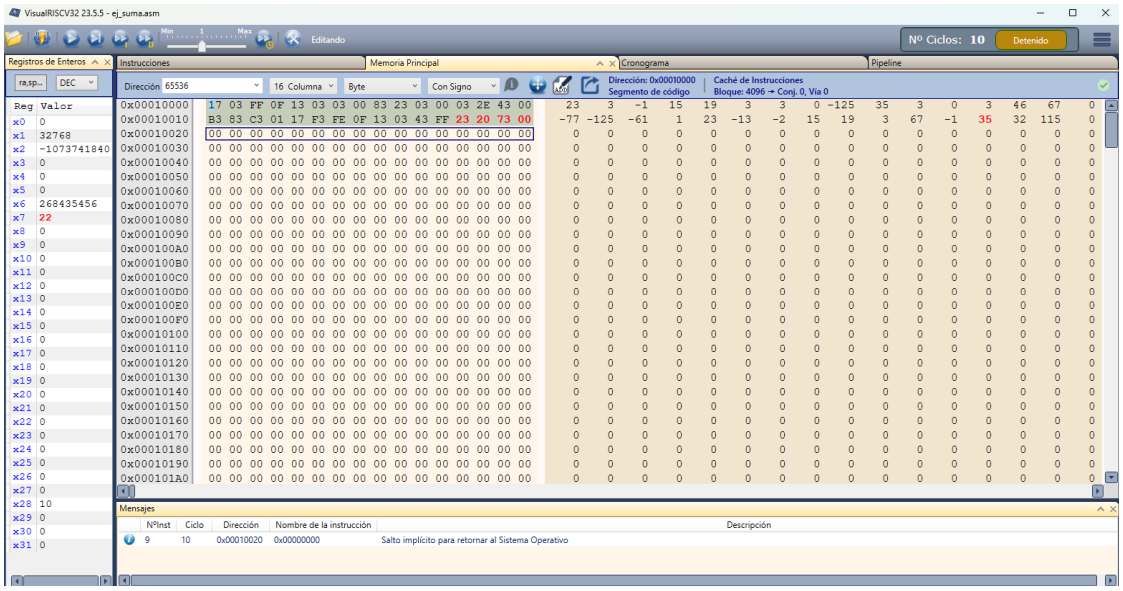


Figura 3

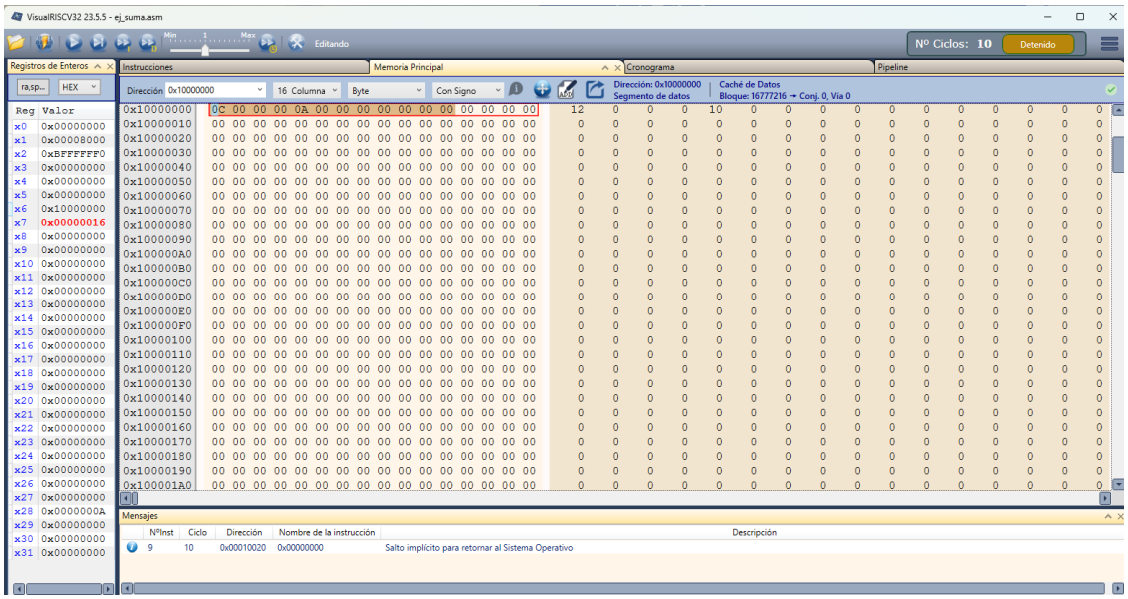


Figura 4

- *Ventana Cronograma*. Muestra la cadena de ejecución de las instrucciones en cada ciclo de reloj. Al ser el RISCV un procesador segmentado, varias instrucciones se ejecutan solapadas en el tiempo. La ejecución de cada instrucción se subdivide en 5 etapas o fases (IF-ID-EX-MEM-WB), de manera que existen 5 instrucciones ejecutándose a la vez pero en etapas diferentes, como se observa en la Figura 5. Haciendo doble *clic* sobre una instrucción puede obtenerse información adicional sobre la instrucción y sus operandos.

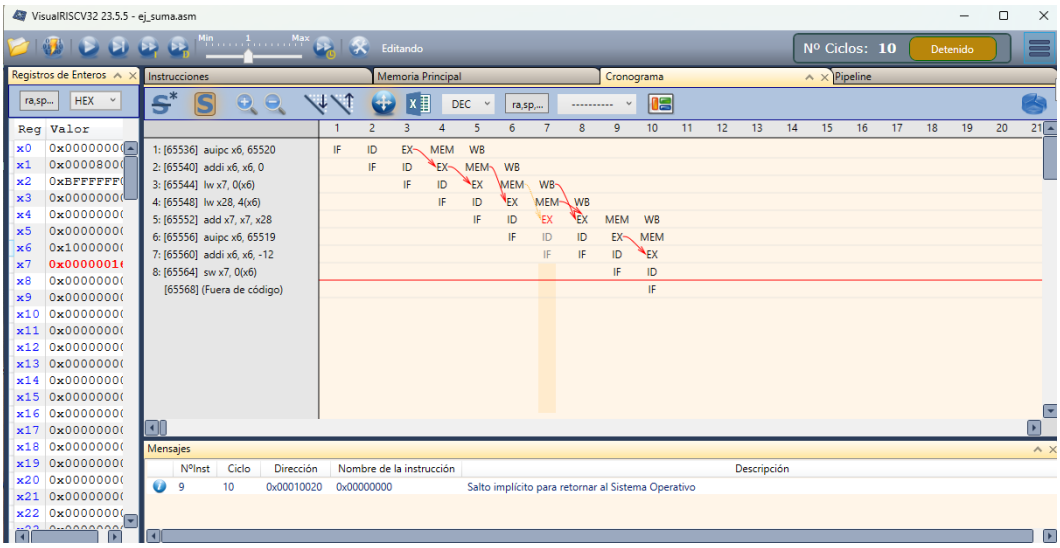


Figura 5

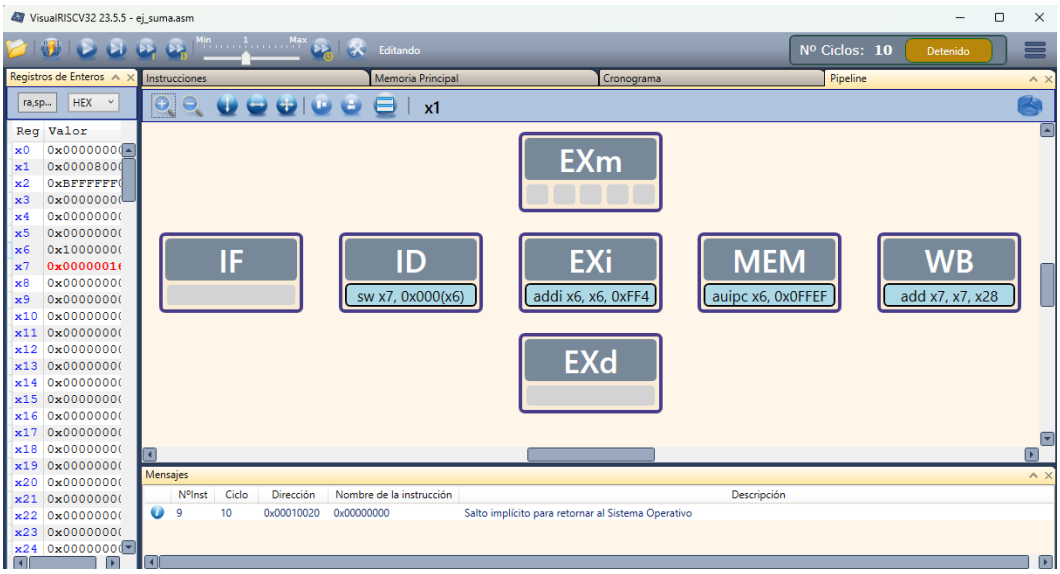


Figura 6

- *Ventana de Pipeline*. Muestra un gráfico de las distintas unidades funcionales de la cadena de ejecución del RISCV tal como se muestra en la Figura 6. En cada una de las unidades funcionales aparece indicada la operación en ejecución. Aparecen las unidades de ejecución para operaciones simples así como las unidades de ejecución de la multiplicación y la división.
- *Ventana de Registro de Enteros*. Muestra el contenido de los 32 registros de enteros. En un procesador segmentado, la depuración y simulación de un programa es una tarea compleja, pues en un mismo ciclo de reloj existen varias instrucciones ejecutándose a la vez (en nuestro caso 5). Por ello, el contenido de los registros internos en un determinado ciclo de reloj depende la etapa en que se encuentre cada instrucción. Por tanto, para entender los valores de los registros que aparecen en la ventana *Register Window* hay que conocer previamente en qué fase está cada instrucción.

Adicionalmente, el simulador VisualRISCV32 permite configurar múltiples opciones en el procesador tales como configuración de segmentación, activación de desvíos, activación de predicción de saltos. La configuración de estas opciones se puede realizar utilizando el menú desplegable en la parte superior derecha de la ventana principal y entrando en el submenú
Archivo->Configuración

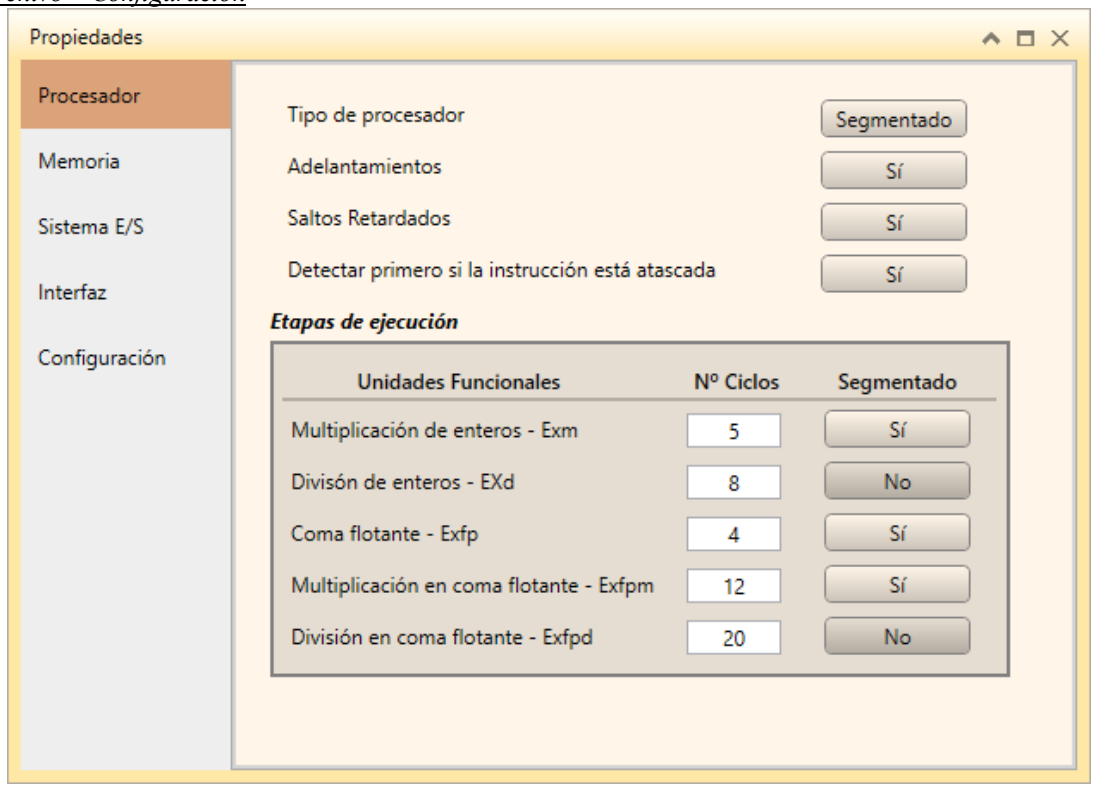
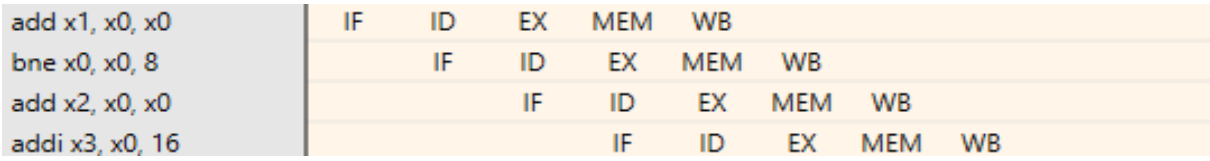
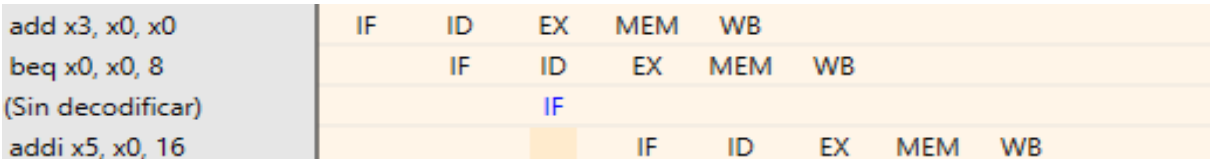


Figura 7

4. BREVE REPASO DE CONCEPTOS TEORICOS BÁSICOS

En esta sesión de laboratorio se estudiarán tanto las dependencias de control como las de datos. Las primeras se dan en las instrucciones de salto. La condición y destino del salto se calculan en la fase ID. La ISA del RISC-V no determina una predicción de saltos concreta. Cada microarquitectura concreta puede definir una predicción de saltos determinada.
El simulador sigue la estrategia de **apostar por salto no tomado**: si el salto finalmente no se toma, la etapa IF ejecutada es válida y, por tanto, no se produce ningún ciclo de bloqueo (Figura superior); sin embargo, si el salto se toma, es necesario ejecutar de nuevo la etapa IF con el destino de salto ya calculado (Figura inferior).





Las dependencias de datos surgen cuando dos instrucciones próximas en el código utilizan datos comunes, ya que la segmentación cambia la temporización relativa de las instrucciones al solapar su ejecución. Existen tres tipos: **dependencias reales** (RAW, lectura después de escritura), **antidependencias** (WAR, escritura después de lectura) y **dependencias de salida** (WAW, escritura después de escritura).
La siguiente figura ilustra un caso de dependencia RAW cuando los adelantamientos no están activos. La instrucción **addi x3,x2,1** lee el registro **x2** antes de que la instrucción anterior haya escrito el valor correcto (en la etapa WB de la primera instrucción se escribe el registro y en la etapa ID de la segunda se lee). Se producen por tanto 2 ciclos de

bloqueo hasta que la etapa ID de la segunda instrucción tiene disponible el valor correcto en la etapa WB de la primera instrucción.

	1	2	3	4	5	6	7	8
addi x2, x1, 1	IF	ID	EX	MEM	WB			
addi x3, x2, 1		IF	ID	ID	ID	EX	MEM	WB

Para evitar los bloqueos producidos por las dependencias de datos (**¡no todos pueden evitarse!**) puede utilizarse la técnica denominada *forwarding* o adelantamiento. Esta técnica consiste en crear caminos entre las unidades funcionales para que las instrucciones dispongan de los datos antes de que se lleve a cabo la etapa WB de la instrucción que los ha calculado. Estos caminos se denominan desvíos o *bypass*. En el simulador VisualRISCV32 los *bypass* se activan en el menú de configuración (tal como se indica en la Figura 7).

La siguiente figura ilustra la misma situación cuando los adelantamientos están activos. La flecha roja ilustra que se produce un adelantamiento de la salida de la etapa EX de la primera instrucción a la entrada de la misma etapa de la segunda instrucción eliminando los dos ciclos de bloqueo.

	1	2	3	4	5	6
addi x2, x1, 1	IF	ID	EX	MEM	WB	
addi x3, x2, 1		IF	ID	EX	MEM	WB

5. DESENROLLADO DE BUCLES Y PLANIFICACIÓN DE INSTRUCCIONES

Algunos bloqueos pueden ser eliminados mediante la **planificación de instrucciones**. Esta técnica consiste en reordenar las instrucciones del código para evitar o reducir el número de bloqueos. En el caso de los bloqueos de datos, estos pueden reducirse si se insertan instrucciones independientes entre los pares de instrucciones que provocan bloqueos. Obviamente, la planificación de instrucciones no debe modificar el resultado del programa. En algunos casos, además de cambiar el orden de las instrucciones, puede ser necesario el renombrado de registros o la modificación de algunas constantes como el *offset* de los operandos en memoria.

Uno de los problemas de la planificación de instrucciones es que, en muchos casos, el programa no dispone de suficientes instrucciones independientes que puedan ser reordenadas. Para evitar este problema y poder disponer de más instrucciones sin dependencias de datos se utiliza la técnica denominada **desenrollado de bucles** (*loop unrolling*).

El desenrollado de bucles consiste replicar el cuerpo del bucle, de forma que una iteración del bucle desenrollado realice las mismas operaciones que k iteraciones del bucle original (el valor k se denomina **factor de desenrollado**). Para que las instrucciones de cada réplica sean independientes a las del resto de réplicas, es necesario renombrar los registros utilizados. Por tanto, esta técnica provoca una mayor utilización de registros. Como consecuencia, el tamaño del banco de registros puede llegar a determinar el número de veces que puede ser replicado el cuerpo del bucle. Por otra parte, es necesario reajustar la actualización del contador del bucle. Por ejemplo, si el contador se incrementa de uno en uno en el bucle original, el incremento debe ser k en el bucle desenrollado. Tras el desenrollado del bucle, debe realizarse la planificación de las instrucciones para reducir los bloqueos.

Además de reducir bloqueos de datos, el desenrollado de bucles tiene el beneficio adicional de reducir el número de instrucciones que se ejecutan para incrementar/decrementar el contador del bucle y el número de instrucciones de salto, lo que permite reducir también el número de bloqueos de control. La inconveniente que presenta el desenrollado es que el código aumenta de tamaño.

El desenrollado de bucles puede aplicarse sin problemas cuando el **bucle es paralelo**; es decir, cuando no presenta dependencias entre iteraciones. Por ejemplo:

```
for (i=0; i<100; i++)
      x[i] = x[i] + 5;
```

Sin embargo, los bucles no paralelos dificultan la aplicación de la técnica. A continuación, se muestra un ejemplo de bucle no paralelo, en el que cada iteración depende de los valores calculados en la iteración anterior:

```
for (i=0; i<100; i++)
      x[i] = x[i-1] + 2*x[i];
```

5.1 Ejemplo de desenrollado de bucles

Supongamos que tenemos el siguiente bucle paralelo, en el que *x* es un vector cuyos elementos ocupan una palabra:

```
for (i=0; i<100; i++)
      x[i] = x[i] + 5;
```

La versión en ensamblador de este bucle podría ser la siguiente:

```
    la      a1, x                 # x base address
    addi    a4, a1, 400           # x address loop end
bucle:
    lw      a2, 0(a1)       # x[i]
    addi    a2, a2, 5       # x[i] + 5
    sw      a2, 0(a1)       # store back
    addi    a1, a1, 4       # i++
    blt     a1, a4, bucle   # loop i<100

    li a7 , 10              # Syscall exit
    ecall
```

Si aplicamos un desenrollado con factor 4 al bucle anterior, el código C equivalente sería el siguiente:

```
for (i=0; i<100; i=i+4) {
      x[i] = x[i] + 5;
      x[i+1] = x[i+1] + 5;
      x[i+2] = x[i+2] + 5;
      x[i+3] = x[i+3] + 5;
}
```

En el código ensamblador sería necesario replicar el grupo de 3 instrucciones que aparecen en negrita. En cada réplica habría que utilizar un *offset* diferente ya que cada una de ellas accede a un elemento diferente del vector. Finalmente, habría que modificar la instrucción que implementa el incremento de la variable *i* dentro del bucle for. El código ensamblador del bucle desenrollado deberá ser programado por el alumno en el Ejercicio 2.

6. EJERCICIOS

Ejercicio 1. Para el código ensamblador que se muestra en la Sección 5.1 realice los siguientes apartados:

a. Dibuje el diagrama de ejecución en un RISCV **sin desvíos** del código ensamblador que se muestra en la Sección 5.1. Dibuje solo la primera iteración y la última. Indique los bloqueos que se producen y, para cada bloqueo, explique de qué tipo es y a qué se debe. Finalmente, compruebe utilizando el simulador VisualRISCV32 (con *Adelantamientos desactivado*) que los resultados obtenidos son correctos.

a. Calcule el CPI (Ciclos por Instrucción) y el BPI (ciclos de Bloqueo por Instrucción) usando las siguientes ecuaciones:

$$CPI = \frac{n_instrucciones_ejecutadas + latencia_inicial + n_bloqueos}{n_instrucciones_ejecutadas}$$

$$BPI = \frac{n_bloqueos}{n_instrucciones_ejecutadas}$$

b. Aplique la planificación de instrucciones para minimizar el número de bloqueos. ¿Cuántos bloqueos de datos pueden eliminarse en cada iteración?

Ejercicio 2. Implemente en ensamblador el bucle desenrollado de la Sección 5.1, pero no aplique la planificación de instrucciones. Dibuje la traza de ejecución en un RISCV **sin desvíos** (dibuje solo la primera iteración).

Ejercicio 3. Aplique la planificación de instrucciones al código del Ejercicio 2 para eliminar el mayor número de bloqueos posible. Dibuje la traza de ejecución en un RISCV **sin desvíos**, indicando los bloqueos que se producen (dibuje solo la primera iteración y la última). Calcule el CPI, el BPI y la aceleración conseguida respecto al código original. ¿Qué bloqueos no han podido ser eliminados? ¿Por qué? Finalmente, compruebe utilizando el simulador RISCV que los resultados obtenidos son correctos.

Ejercicio 4. Repita los Ejercicios 1, 2 y 3 suponiendo un RISCV con todos los desvíos activos. Para cada desvío indique de qué registro de segmentación o unidad funcional se toma el dato y a qué registro de segmentación o unidad funcional se lleva.
¿Qué conclusiones se obtienen por el hecho de utilizar desvíos? ¿Cuántos bloqueos de cada tipo se han eliminado? Finalmente, compruebe utilizando el simulador RISCV (con *Adelantamientos* activado) que los resultados obtenidos son correctos.

Ejercicio 5. Sea el siguiente fragmento de código, que convierte una cadena de caracteres en mayúscula a minúscula:

```
.data

cadena: .asciiz "CADENAENMAYUSCULAS"

.text
       la    a1,  cadena
 cont: lb    a2, 0(a1)
       beqz  a2, FIN
       addi  a2, a2, 32
       sb    a2 , 0(a1)
       addi  a1, a1, 1
       j     cont
 FIN:
       li a7 , 10  # Syscall exit
       ecall
```

a. Dibuje en papel el diagrama de ejecución para un RISCV sin ningún tipo de desvío. Dibuje solo la primera iteración y la última. ¿En cuántos ciclos se ejecuta este bucle? Para dibujar las trazas puede utilizar las plantillas que se suministran al final del boletín.

b. Repita el apartado (a) suponiendo que el RISCV tiene todos los desvíos posibles.

c. Aplique la planificación de instrucciones para minimizar el número de bloqueos y repita el apartado (a) en un RISCV sin desvíos activos.

d. Repita el apartado (c) suponiendo que el RISCV tiene todos los desvíos posibles.